

Lecture 2: Intro to C

Professor G. Sandoval

02



Some slides adapted by G. Sandoval from slide
by Brendan Dolan-Gavitt

Learn C in 100 minutes

- Just kidding, it takes longer than that!
- But we'll cover the basics in class today
- For more details, you can consult:
 - Tutorial: Beej's guide to C programming: <https://beej.us/guide/bgc/>
 - Reference: <https://en.cppreference.com/w/c.html>
 - Best C meets OS book: <https://www.amazon.com/Computer-Systems-Programmers-Perspective-3/dp/9332573905>
 - The OG: Brian Kernighan and Dennis Ritchie, The C Programming Language <https://archive.org/details/TheCProgrammingLanguage>
 - Fun intro: <https://www.amazon.com/Head-First-C-Brain-Friendly-Guide/dp/1449399916>

A First Program

Include file: similar to
“import” in Python

```
1  #include <stdio.h>
2
3  int main()
4  {
5      printf("hello world\n");
6      return 0;
7  }
```

A First Program

```
1  #include <stdio.h>
2
3  int main()
4  {
5      printf("hello world\n");
6      return 0;
7  }
```

Declaration of function
“main”, returning type “int”

A First Program

```
1  #include <stdio.h>
2
3  int main()
4  {
5      printf("hello world\n");
6      return 0;
7  }
```

Calling the printf function
with a string parameter

A First Program

```
1  #include <stdio.h>
2
3  int main()
4  {
5      printf("hello world\n");
6      return 0;
7  }
```

Each statement must end
with a semicolon

A First Program

```
1  #include <stdio.h>
2
3  int main()
4  {
5      printf("hello world\n");
6      return 0;
7  }
```

Return the value 0

Compiling and Running



The Compilation Process

Invoking gcc on hello.c did the following things:

- a. Called the C *preprocessor* (cpp) on hello.c
- b. cpp looked for stdio.h in the default include path (a set of directories where header files live) and then recursively called itself on stdio.h
- c. The output of that process was pasted into hello.c, and the result was compiled by gcc into an executable binary

Aside: Makefiles

- Remembering and reproducing the commands to compile a project is tedious
- Larger projects may have multiple files, many compilation options, external libraries, etc.
- To manage this complexity we can use a Makefile

make

- Instead of calling gcc, we can instead just do:

```
$ make hello
```

```
cc      hello.c  -o hello
```

- The make command has some built-in rules that understand how to compile basic C and C++ programs
- The make command will check whether the file "hello" exists and is newer than the file "hello.c"; if not, it will try to build "hello" using "hello.c"

Back to C

New program:

- a. Write a C program that prints:
 - the parameters you passed and
 - the number of parameters.
- b. Compile and run it!

Back to C: Declaring Variables

```
1  #include <stdio.h>
```

```
2
```

```
3  int main() {
```

```
4  int height = 70;
```

```
5  int age;
```

```
6  age = 48;
```

```
7  printf("I'm %d years old and %d inches tall", age, height);
```

```
8  return 0;
```

```
9  }
```

Variable Type

Initial Value

Variable Name

Basic Data Types

char	A single byte, capable of holding one character in the local character set	0 – 255 Or -128 to 127
int	An integer, typically reflecting the natural size of integers in the local machine	-2147483648 To 2147483647
float	Single precision floating point	32 bit IEEE floating point
Double	Double precision floating point	32 bit IEEE floating point

Note: some of these ranges are different on different platforms and implementations of C. Here I'm giving the ranges for 32-bit x86, which is what we'll be working with this semester. More precise details can be found by consulting the C language specification.

C Operators

- **Arithmetic:**
 - + (plus), - (minus), / (division), * (multiplication), % (modulus)
- **Logical**
 - && (and), || (or), ! (not)
- **Relational**
 - < (less than), > (greater than), >= (greater or equal), <= (less or equal),
== (equal), != (not equal)

C Operators

Arithmetic:

- Binary arithmetic
- $\sim x$: one's complement (i.e., flip all bits)
- $x \ll n$: shift x left by n bits
- $x \gg n$: shift x right by n bits
- $x \& y$: bitwise AND of x and y
- $x | y$: bitwise OR of x and y
- $x \wedge y$: bitwise XOR of x and y

Control Flow

Branching:

```
if (condition) {  
    statements  
}  
  
else if (condition) {  
    statements  
}  
  
else {  
    statements  
}
```

Control Flow

Branching:

```
switch (expression) {  
    case 1:  
        statements  
        break;  
    case 2:  
        statements  
        break;  
    default:  
        statements  
        break;  
}
```

Control Flow

Looping:

```
while (condition) {  
    statements  
}  
  
do {  
    statements  
} while (condition);
```

Control Flow

Looping:

```
for (init ; condition ; update) {  
    statements  
}
```

Example:

```
int i;  
  
for (i = 0; i < 100; i = i + 1) {  
    printf("This is iteration %d\n", i);  
}
```

Putting It Together

Breakout:

"Write a program that prints the numbers from 1 to 100. But for multiples of three print "Fizz" instead of the number and for the multiples of five print "Buzz". For numbers which are multiples of both three and five print "FizzBuzz"."

Functions

- All code in C lives in a function
- Declaring a function is simple:

```
int add(double x, double y) {  
    return x + y;  
}
```

- Calling it is simple too

```
int result = add(4.9, 12.1);
```

Functions

- All code in C lives in a function
- Declaring a function is simple:

```
int add(double x, double y) {  
    return x + y;  
}
```

- Calling it is simple too

```
int result = add(4.9, 12.1);
```

Return Type

Function name

Parameters

Assignment Operators

- $x = y$: simple assignment

Note: $x = y$ is an **expression**, and evaluates to the **right hand side**. So, you can do things like

```
if ((x = y) > 0) ...
```

which will assign x the value of y and branch if y is greater than 0

- $x += y$: same as $x = x + y$
- $-=$, $*=$, $/=$, $<<=$, $>>=$, $\%=$, $\&=$, $|=$

Increment example

```
1  #include <stdio.h>
2  int main() {
3      int x = 0;
4      if (x++ > 0) {
5          printf(..)
6          // A will this be printed ?
7      }
8      if (x > 0) {
9          printf(. .)
10         // B will this be printed ?
11     }
12     x = 0;
13     if (++x > 0) {
14         printf(..)
15         // C will this be printed ?
16     }
17     if (x > 0) {
18         1 Printf(..)
19         // D will this be printed ?
20     }
21     return 0;
22 }
```

Class 2: What will be printed?

Only B and D

0%

Only A and C

0%

B, C and D

0%

Nothing

0%

More C Operators

- (Pre/Post) Increment/decrement
 - `x++`: evaluate `x`, then increment it
 - `++x`: increment `x`, then evaluate it
 - `x--`: evaluate `x`, then decrement it
 - `--x`: decrement, then evaluate it

Increment example

```
1  #include <stdio.h>
2  int main() {
3      int x = 0;
4      if (x++ > 0) {
5          printf("This will not be printed\n");
6      }
7      if (x > 0) {
8          printf("This will be printed\n");
9      }
10     x = 0;
11     if (++x > 0) {
12         printf("This will be printed\n");
13     }
14     if (x > 0) {
15         printf(" This will be printed \n");
16     }
17     return 0;
18 }
```

Arrays

- Arrays in C are *fixed-length* (their size must be specified at compile time)
- To declare an array:
`int x[10];`
- Then individual elements are accessible as `x[i]` where `i` is an integer

Array Bounds

- Unlike higher-level languages, C **does not check** array bounds
 - In some simple cases, the compiler can detect when you access an out-of-bounds element, but it's not part of the language
- Programming in C requires you to be very careful when programming; there are basically **no safety features**

What happens when we run this code?

```
1  #include <stdio.h>

2  int main() {
3      int x[5];
4
5      x[10]=9;
6
7      print("x[10]=%d\n",x[10]);
8      return 0;
9  }
```

Segfaults

```
cosimo:lec2 moyix$ gcc oob.c -o oob
```

```
oob.c:6:5: warning: array index 10 is past the end of the array  
(which contains 5 elements) [-Warray-bounds]
```

```
x[10] = 9;
```

```
^ ~~
```

```
oob.c:4:5: note: array 'x' declared here
```

```
int x[5];
```

```
^
```

```
oob.c:8:28: warning: array index 10 is past the end of the array  
(which contains 5 elements) [-Warray-bounds]
```

```
printf("x[10] = %d\n", x[10]);
```

```
^ ~~
```

```
oob.c:4:5: note: array 'x' declared here
```

```
int x[5];
```

```
^
```

2 warnings generated.

```
cosimo:lec2 moyix$ ./oob
```

```
x[10] = 9
```

```
Segmentation fault: 11
```

Class 2: What happens when we run OOB.c

Prints 9 with no error

0%

Prints random garbage value

0%

It causes a segmentation fault

0%

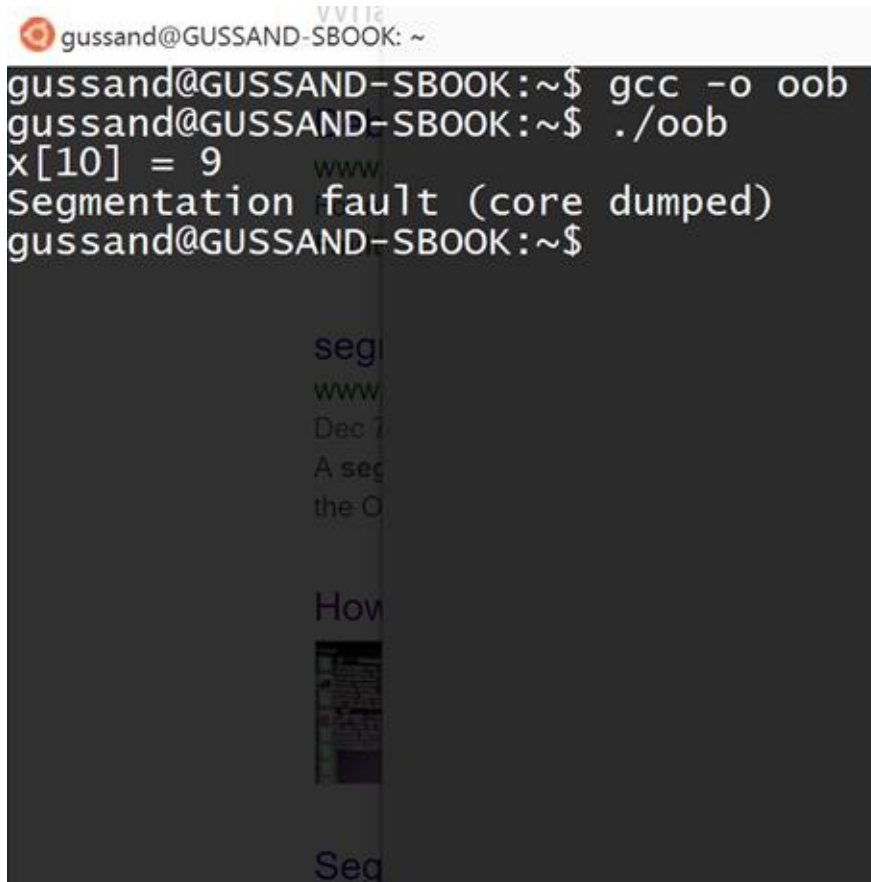
Undefined behavior. Any of the above is possible

0%

None of the above

0%

segfaults



```
gussand@GUSSAND-SBOOK: ~  
gussand@GUSSAND-SBOOK:~$ gcc -o oob oob.c  
gussand@GUSSAND-SBOOK:~$ ./oob  
x[10] = 9  
Segmentation fault (core dumped)  
gussand@GUSSAND-SBOOK:~$
```

43

Array Bounds

44

OOB Example

45

Segfaults

46

segfaults

47

What About Strings?

What About Strings?

- Strings in C are a **special case of arrays**
- A C string is an **array of characters**, where the **final element** in the array is **NULL** (`'\0'`)
- So:

```
char hello[] = "hi";
```

- is the same as

```
char hello[] = {'h', 'i', '\0'};
```

What will this program print?

```
1  #include <stdio.h>

2  int main() {
3      int x=10;
4      int *p = &x;
5
6      printf("%p \n", p);
7      return 0;
8  }
```

Class 2: What will this program print?

The value 10

0%

A memory adress like (0x16bc)

0%

NULL

0%

Compile Error

0%

None of the above

0%

Pointers

- C has a special kind of variable called a *pointer*
- Pointers hold the *address* of another variable

- To declare:

```
int *foo;
```

- Two new operators:

&x : get the *address of* x

***p** : access the *value of* the pointed-to variable
(called *dereferencing*)

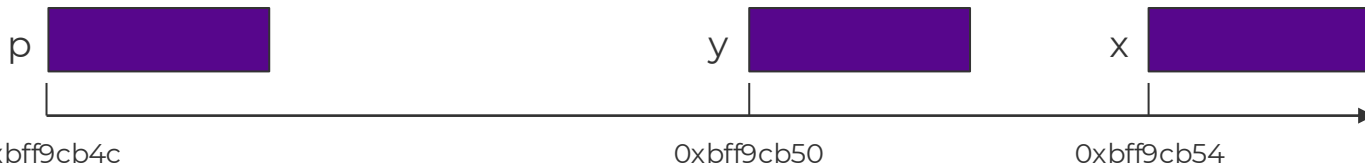
Pointers

Output

```
1  #include <stdio.h>

2  int main() {
3      int *p = NULL;
4      int y = 20;
5      int x = 10;

6      printf("At the start, x = %d, y = %d, p = %p\n", x, y, p);
7      p = &x;
8      printf("Pointer at %p has value %p and points to var with\n", &p, p, *p);
9          value %d\n", &p, p, *p);
10     *p = 5;
11     printf("After assigning to *p, x = %d\n", x);
12     p = &y;
13     printf("Pointer at %p has value %p and points to var with\n", &p, p, *p);
14         value %d\n", &p, p, *p);
15     *p = 50;
16     printf("After assigning to *p, y = %d\n", y);
17     return 0;
18 }
```

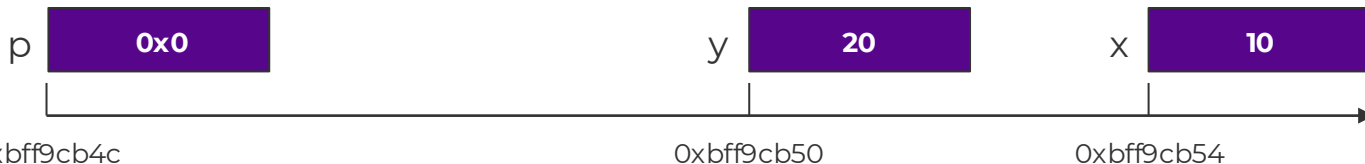


Pointers

```
1  #include <stdio.h>
2  int main() {
3      int *p = NULL;
4      int y = 20;
5      int x = 10;
6
7      printf("At the start, x = %d, y = %d, p = %p\n", x, y, p);
8      p = &x;
9      printf("Pointer at %p has value %p and points to var with\n", &p, p, *p);
10     value %d\n", &p, p, *p);
11     *p = 5;
12     printf("After assigning to *p, x = %d\n", x);
13     p = &y;
14     printf("Pointer at %p has value %p and points to var with\n", &p, p, *p);
15     value %d\n", &p, p, *p);
16     *p = 50;
17     printf("After assigning to *p, y = %d\n", y);
18     return 0;
19 }
```

Output

At the start, x = 10, y = 20, p = 0x0



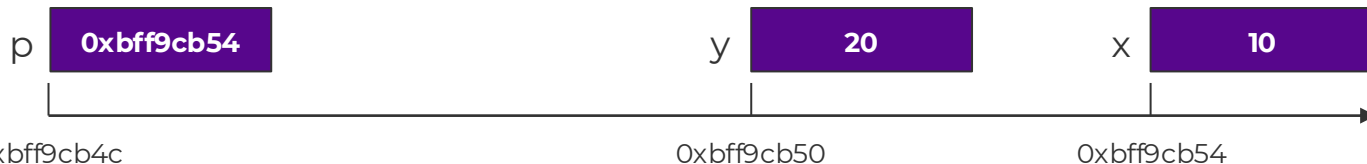
Pointers

```
1  #include <stdio.h>
2  int main() {
3      int *p = NULL;
4      int y = 20;
5      int x = 10;
6      printf("At the start, x = %d, y = %d, p = %p\n", x, y, p);
7      p = &x;
8      printf("Pointer at %p has value %p and points to var with\n", &p, p, *p);
9          value %d\n", &p, p, *p);
10     *p = 5;
11     printf("After assigning to *p, x = %d\n", x);
12     p = &y;
13     printf("Pointer at %p has value %p and points to var with\n", &p, p, *p);
14         value %d\n", &p, p, *p);
15     *p = 50;
16     printf("After assigning to *p, y = %d\n", y);
17     return 0;
18 }
```

Output

At the start, x = 10, y = 20, p = 0x0

Pointer at 0xbff9cb4c
has value 0xbff9cb54 and
points to var with value 10



Pointers

```
1  #include <stdio.h>

2  int main() {
3      int *p = NULL;
4      int y = 20;
5      int x = 10;

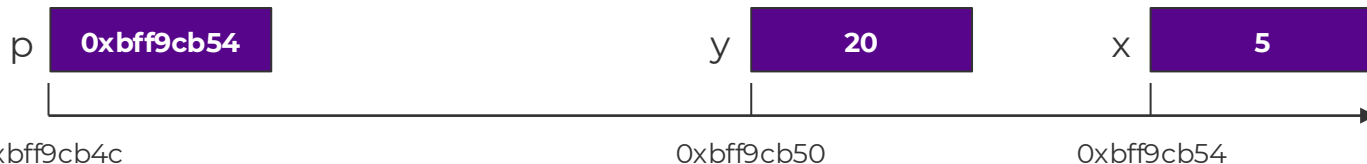
6      printf("At the start, x = %d, y = %d, p = %p\n", x, y, p);
7      p = &x;
8      printf("Pointer at %p has value %p and points to var with\n",
9             value %d\n", p, p, *p);
10     *p = 5;
11     printf("After assigning to *p, x = %d\n", x);
12     p = &y;
13     printf("Pointer at %p has value %p and points to var with\n",
14            value %d\n", &p, p, *p);
15     *p = 50;
16     printf("After assigning to *p, y = %d\n", y);
17     return 0;
18 }
```

Output

At the start, x = 10, y = 20, p = 0x0

Pointer at 0xbff9cb4c
has value 0xbff9cb54 and
points to var with value 10

After assigning to *p, x = 5



Pointers

```
1  #include <stdio.h>

2  int main() {
3      int *p = NULL;
4      int y = 20;
5      int x = 10;

6      printf("At the start, x = %d, y = %d, p = %p\n", x, y, p);
7      p = &x;
8      printf("Pointer at %p has value %p and points to var with\n",
9             value %d\n", &p, p, *p);
10     *p = 5;
11     printf("After assigning to *p, x = %d\n", x);
12     p = &y;
13     printf("Pointer at %p has value %p and points to var with\n",
14            value %d\n", &p, p, *p);
15     *p = 50;
16     printf("After assigning to *p, y = %d\n", y);
17     return 0;
18 }
```

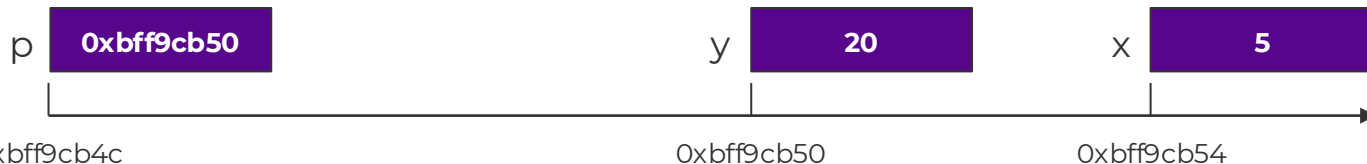
Output

At the start, x = 10, y = 20, p = 0x0

Pointer at 0xbff9cb4c
has value 0xbff9cb54 and
points to var with value 10

After assigning to *p, x = 5

Pointer at 0xbff9cb4c
has value 0xbff9cb50 and
points to var with value 20



Pointers

```
1  #include <stdio.h>

2  int main() {
3      int *p = NULL;
4      int y = 20;
5      int x = 10;

6      printf("At the start, x = %d, y = %d, p = %p\n", x, y, p);
7      p = &x;
8      printf("Pointer at %p has value %p and points to var with\n",
9             value %d\n", &p, p, *p);
10     *p = 5;
11     printf("After assigning to *p, x = %d\n", x);
12     p = &y;
13     printf("Pointer at %p has value %p and points to var with\n",
14            value %d\n", &p, p, *p);
15     *p = 50;
16     printf("After assigning to *p, y = %d\n", y);
17     return 0;
18 }
```

Output

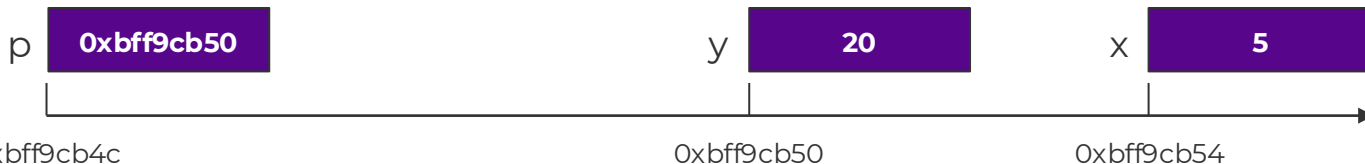
At the start, x = 10, y = 20, p = 0x0

Pointer at 0xbff9cb4c
has value 0xbff9cb54 and
points to var with value 10

After assigning to *p, x = 5

Pointer at 0xbff9cb4c
has value 0xbff9cb50 and
points to var with value 20

After assigning to *p, y = 50



Practice 1: Count newlines in a chunk

Function Definition:

```
int count_newlines(const char *buf, int n);
```

Inputs

- buf points to a sequence of **bytes** (often from read).
- n is the number of bytes that are valid to examine.

Behavior

- Return the number of newline characters ('\n') that appear in the **first n bytes** starting at buf.
- Treat the buffer as **raw bytes**, not as a null-terminated string:
- Do **not** stop at '\0'.
- Do **not** call strlen.
- If $n \leq 0$, return 0.

Practice 1 (cont)

- **Constraints / assumptions**

- buf is non-NULL when $n > 0$
- Runs in $O(n)$ time, $O(1)$ extra space.

Example

- `buf = "a\nb\nc"`, $n = 5 \rightarrow$ returns 2
- `buf = {'x','\0','\n'}`, $n = 3 \rightarrow$ returns 1

Call by Value vs Call by Reference

- Call-by-value: Changes made to arguments passed to a function aren't reflected in the calling function
- Call-by-reference: Changes made to arguments passed to a function are reflected in the calling function
- **C is a call-by-value language**
- To cause changes to values outside the function, **use pointers**
 - Do not assign the pointer to a different value (that won't be reflected!)
 - Instead, dereference the pointer and assign a value to that address

Call by Value vs Call by Reference

Example:

- Write a function called swap that given two integers it swaps it's values:

Call by Value vs Call by Reference

Example:

- Write a function called swap that given two integers it swaps it's values:

```
int x = 42;  
int y = 54;  
swap (&x, &y);  
printf("%d\n", x); // 54  
printf("%d\n", y); // 42
```

Call by Value vs Call by Reference

Example:

- Write a function called swap that given two integers it swaps it's values:

```
void swap(int* a, int* b) {  
    Int temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

```
int x = 42;  
int y = 54;  
swap (&x, &y);  
printf("%d\n", x); // 54  
printf("%d\n", y); // 42
```

Converting

- Variables of different types can be converted to one another

- For example:

```
int x = 10;
float y = x;
printf("x = %d, y = %f\n", x, y);
```

Prints

```
x = 10, y = 10.000000
```

- The details of this are somewhat complicated, but a general rule is that things can be converted automatically as long as you aren't losing precision

Casting Types

- Some conversions need an explicit cast – a way of telling C that you really do want that conversion
- For example:

```
int x = 90000;  
short y = (short) x;  
printf("x = %d, y = %d\n", x, y);
```

Prints

```
x = 90000, y = 24464
```

Casting

- Pointers of one type can be cast to another type
- As usual, C does not stop you from shooting yourself in the foot with this
- But it does let you do some handy things too!

Casting

```
1  #include <stdio.h>

2  int main() {
3      char x[] = "POLY";
4      int *y;
5      y = (int *) &x;
6      printf("%s is %d\n", x, *y);
7      return 0;
8  }
```

Output

POLY is 1498173264

Casting

```
1  #include <stdio.h>

2  int main() {
3      char x[] = "POLY";
4      int *y;
5      y = (int *) &x;
6      printf("%s is %d\n", x, *y);
7      return 0;
8  }
```

Output

POLY is 1498173264

Why

Casting

```
1  #include <stdio.h>

2  int main() {
3      char x[] = "POLY";
4      int *y;
5      y = (int *) &x;
6      printf("%s is %d\n", x, *y);
7      return 0;
8  }
```

Output

POLY is 1498173264

Why

"POLY" = {'P', 'O', 'L', 'Y', '\0'}
= 0x50, 0x4f,
0x4c, 0x59, 0x00
= 0x594c4f50
= 1498173264

Casting (Other Direction)

```
1  #include <stdio.h>

2  int main() {
3      int y = "1498173264";
4      char x[5];
5      *((int *)x) = y;
6      x[4] = '\\0';
7      printf("%d is %s\\n", y, x);
8      return 0;
9  }
```

Output

1498173264 is POLY

Structures

- Sometimes we want more complicated data types than just int, char, etc.
- For this we can define a *struct* – an aggregate type that contains several fields

Data Structures

```
1  #include <stdio.h>

2  struct Person {
3      int age;
4      int height_cm;
5      int weight_kg;
6  };

7  int main() {
8      int i;
9      Struct Person p[2];
10
11     p[0].age = 60;
12     p[0].height_cm = 150;
13     p[0].weight_kg = 90;
14
15     p[1].age = 21;
16     p[1].height_kg = 180;
17     p[1].weight_kg = 80;
18
19     for (i = 0; i < 2; i++) {
20         printf("Person %d is %d years old and weighs %d kg \n",
21             i, p[i].age, p[i].weight_kg);
22     }
23     return 0;
24 }
```

One Last Operator

- If you have a pointer to a structure, you can dereference the pointer and access its member in one step:

```
struct Person *p = &q;
```

```
p->age = 64;
```

Arrays and Pointers

- Arrays and pointers have a special relationship in C
- An array can be treated as a pointer in most contexts, and vice versa
- If you hand an array of type T to something that expects a pointer to type T, it's treated as if it were a pointer to the first element of the array

Pointer Arithmetic

- You can do arithmetic on pointers
- Given a pointer P of type T, $P + N$ will point to the memory at address $P + (N * \text{sizeof}(T))$
- For example:

```
1  int x [4] = {1, 2, 3, 4};
2  int *p = x;
3  p = p + 2;
4  printf("*p = %d\n", *p);
```
- => Prints **"*p = 3"**

Pointer Arithmetic vs Array Indexing

- You may have noticed that `x[2]` and `*(p + 2)` refer to the same element
- This is not an accident!
- In fact, we can use array index notation with pointers as well:

```
1  int x [4] = {1, 2, 3, 4};  
2  int *p = x;  
3  printf("p[2] = %d\n", p[2]);
```

- => Prints `"p[2] = 3"`

Memory Allocation

- Recall that I said C only supports fixed-length arrays
- So how do you deal with variable amounts of data?
- To allocate: **p = malloc(size in bytes)** – returns a pointer to a memory region that you can then assign to a variable of whatever type you like
- To free: **free(p)**

Malloc and Free

- **Handle dynamic memory**
- `void* malloc (size_t size):`
 - allocate block of memory of `size` bytes
 - does not initialize memory
- `void free (void* ptr):`
 - frees memory block, previously allocated by `malloc`, `calloc`, `realloc`, pointed by `ptr`
 - use exactly once for each pointer you allocate
- size **argument**:
 - *should* be computed using the **sizeof** operator
 - `sizeof`: takes a type and gives you its size
 - e.g., `sizeof(int)`, `sizeof(int*)`

Memory Management Rules

- **Malloc what you free, free what you malloc**
 - client should free memory allocated by client code
 - library should free memory allocated by library code
- **Number mallocs = Number frees**
 - Number mallocs > Number Frees: definitely a memory leak
 - Number mallocs < Number Frees: definitely a double free
- **Free a malloced block exactly once**
 - Should not dereference a freed memory block
- **Don't put too much data on malloced memory**
 - Make sure you allocate extra space for NULL on strings

Stack Vs Heap Allocation

- Local variables and function arguments are placed on the **stack**
 - deallocated after the variable leaves **scope**
 - **do not return** a pointer to a stack-allocated variable!
 - **do not reference** the address of a variable outside its scope!
- Memory blocks allocated by calls to malloc/calloc are placed on the **heap**
- Globals, constants are placed elsewhere
- Example:
 - // a is a pointer on the stack to a memory block on the heap
 - `int* a = malloc(sizeof(int));`

Example: Linked List

```
1  struct list {  
2      int data;  
3      struct list *next;  
4  };
```

Insertion: What's the pseudocode?

```
1  struct list*list_insert(struct list *head, int data) {  
2      /* Write code for insert */  
3  }
```

Insertion

```
1  struct list * list_insert(struct list *head, int data) {  
2      struct list *new = malloc(sizeof(struct list));  
3      new->data = data;  
4      new->next = head;  
5      return new;  
6  }
```

Insertion

```
1  struct list * list_insert(struct list *head, int data) {  
2      struct list *new = malloc(sizeof(struct list));  
3      new->data = data;  
4      new->next = head;  
5      return new;  
6  }
```

Memory allocation

sizeof operator gets the
size of a type

Insertion

```
Void list_delete(struct list *head) {  
    /* Write code for freeing all the nodes on the list and */  
}
```

Example: Linked List

```
1  int main() {
2      int i;
3      struct list *head = NULL;
4      struct list *cursor = NULL;
5      //Add some stuff to our list
6      for (i = 0; i < 20; i++) {
7          head = list_insert(head, i * 2);
8      }
9      //Now print it out
10     cursor = head;
11     while (cursor != NULL) {
12         printf("list entry at %p has data %d\n",
13             cursor, cursor->.data);
14         cursor = cursor->next;
15     }
16     //And free everything
17     cursor = head;
18     while (cursor != NULL) {
19         struct list *to_delete = cursor;
20         cursor = cursor->next;
21         free(to_delete);
22     }
23 }
```

Example: Linked List

```
1  int main() {
2      int i;
3      struct list *head = NULL;
4      struct list *cursor = NULL;
5      //Add some stuff to our list
6      for (i = 0; i < 20; i++) {
7          head = list_insert(head, i * 2);
8      }
9      //Now print it out
10     cursor = head;
11     while (cursor != NULL) {
12         printf("list entry at %p has data %d\n",
13             cursor, cursor->data);
14         cursor = cursor->next;
15     }
16     //And free everything
17     cursor = head;
18     while (cursor != NULL) {
19         struct list *to_delete = cursor;
20         cursor = cursor->next;
21         free(to_delete);
22     }
23 }
```

↑

Freeing memory

Output

List entry at 0x7b6517d0 has data 38
List entry at 0x7b6517c0 has data 36
List entry at 0x7b6517b0 has data 34
List entry at 0x7b6517a0 has data 32
List entry at 0x7b651790 has data 30
List entry at 0x7b651780 has data 28
List entry at 0x7b651770 has data 26
List entry at 0x7b651760 has data 24
List entry at 0x7b651750 has data 22
List entry at 0x7b651740 has data 20
List entry at 0x7b651730 has data 18
List entry at 0x7b651720 has data 16
List entry at 0x7b651710 has data 14
List entry at 0x7b651700 has data 12
List entry at 0x7b6516f0 has data 10
List entry at 0x7b6516e0 has data 8
List entry at 0x7b6516d0 has data 6
List entry at 0x7b6516c0 has data 4
List entry at 0x7b6516b0 has data 2
List entry at 0x7b6516a0 has data 0

Example: Linked List

Why did we have to use the
extra `to_delete` variable?

```
1  int main() {
2      int i;
3      struct list *head = NULL;
4      struct list *cursor = NULL;
5      //Add some stuff to our list
6      for (i = 0; i < 20; i++) {
7          head = list_insert(head, i * 2);
8      }
9      //Now print it out
10     cursor = head;
11     while (cursor != NULL) {
12         printf("list entry at %p has data %d\n",
13             cursor, cursor->data);
14         cursor = cursor->next;
15     }
16     //And free everything
17     cursor = head;
18     while (cursor != NULL) {
19         struct list *to_delete = cursor;
20         cursor = cursor->next;
21         free(to_delete);
22     }
23 }
```

Things Not Covered

- Function pointers
- Code in multiple files
- Linking to external libraries
- Preprocessor macros
- Some modifiers: static, const, volatile, extern
- Ternary operator, variable-argument functions (e.g., printf)

To Summarize

- C is a very low level language. You can address memory directly.
- We saw a lot of great features of C
- We saw a lot of ways you can shoot yourself in the foot with C
- You will need to know/learn C for this class.

Is something wrong?

```
1  int main() {  
2      char w[strlen("C programming")];  
3      strcpy(w, "C programming");  
4      printf("%s\n", w);  
5      return 0;  
6  }
```


Is something wrong?

```
1  int main() {  
2      char w[strlen("C programming")];  
3      strcpy(w, "C programming");  
4      printf("%s\n", w);  
5      return 0;  
6  }
```

strlen returns the length of the string **not including the null character**, so we end up writing a null byte outside the bounds of `w`.

2. Something Wrong?

```
1 char *strcdup(int n, char c) {
2   char dup[n+1];
3   int i;
4   for (i = 0; i < n; i++)
5     dup[i] = c;
6   dup[i] = '\0';
7   char *A = dup;
8   return A;
9 }
```


2. Something Wrong?

```
1 char *strcdup(int n, char c) {
2   char dup[n+1];
3   int i;
4   for (i = 0; i < n; i++)
5     dup[i] = c;
6   dup[i] = '\\0';
7   char *A = dup;
8   return A;
9 }
```

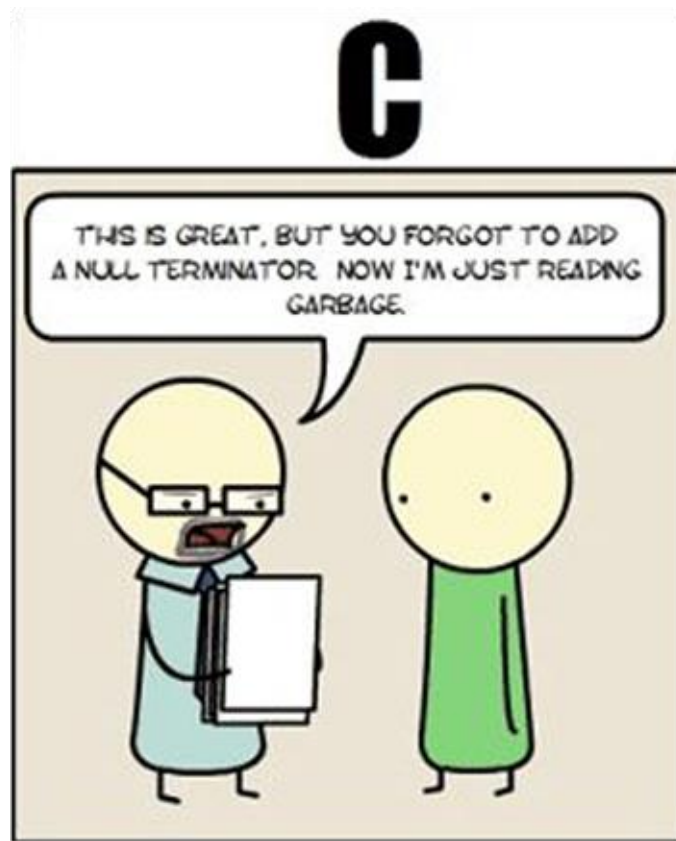
strcdup returns a stack-allocated pointer. The contents of A will be unpredictable once the function returns.



C Libraries

string.h: Common String/Array Methods

- One the most useful libraries available to you
- Important usage details regarding arguments:
 - prefixes: **str** -> strings, **mem** -> arbitrary memory blocks.
 - ensure that all strings are `'\0'` terminated!
 - ensure that **dest** is large enough to store **src**!
 - ensure that **src** actually contains *n* bytes!
 - ensure that **src/dest** don't overlap!



string.h: Common String/Array Methods

- **Copying:**

- `void* memcpy (void* dest, void* src, size_t n)` : copy n bytes of src into dest, return dest
- `char* strcpy(char* dest, char* src)` : copy src string into dest, return dest

- **Concatenation:**

- `char* strcat (char* dest, char* src)` : append copy of src to end of dest, return dest

- **Comparison:**

- `int strcmp (char* str1, char* str2)` : compare str1, str2 by character (based on ASCII value of each character, then string length), return comparison result
str1 < str2: -1,
str1 == str2: 0,
str1 > str2: 1

string.h: Common String/Array Methods (Continued)

- **Searching:**

- `char* strstr (char* str1, char* str2)`: return pointer to *first* occurrence of `str2` in `str1`, else `NULL`
- `char* strtok (char* str, char* delimiters)`: tokenize `str` according to delimiter characters provided in `delimiters`, return the next token per successive stroke call, using `str = NULL`

- **Other:**

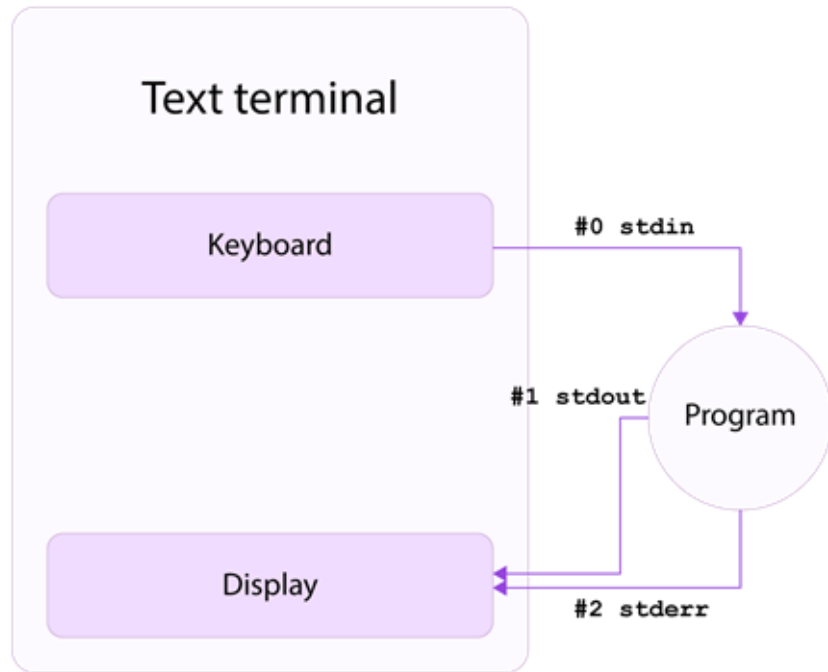
- `size_t strlen ( const char* str )`: returns length of the string (up to, but not including the `'\0'` character)
- `void* memset (void* ptr, int val, size_t n )`: set first `n` bytes of memory block addressed by `ptr` to `val` (use this for *setting bytes* only; don't use to set `int` arrays or anything else!)

stdlib.h: General Purpose Functions

- **Dynamic memory allocation:**
 - `malloc`, `calloc`, `free`
- **String conversion:**
 - `int atoi(char* str)`: parse string into integral value (return 0 if not parsed)
- **System Calls:**
 - `void exit(int status)`: terminate calling process, return **status** to parent process
 - `void abort()`: aborts process abnormally
- **Searching/Sorting:**
 - provide array, array size, element size, comparator (function pointer)
 - `bsearch`: returns pointer to matching element in the array
 - `qsort`: sorts the array destructively
- **Integer arithmetic:**
 - `int abs(int n)`: returns absolute value of **n**
- **Types:**
 - `size_t`: unsigned integral type (store size of *any* object)

stdio.h

- Another really useful library.
- Used for:
 - Argument parsing
 - File handling
 - Input/output



stdio.h: Common I/O Methods

- **FILE* fopen (char* filename, char* mode):** open the file with specified filename in specified **mode** (read, write, append, etc), associate it with stream identified by returned file pointer
- **int fscanf (FILE* stream, char* format, ...):** read data from the **stream**, store it according to the parameter **format** at the memory locations pointed at by additional arguments.
- **int fclose (FILE* stream):** close the file associated with the **stream**
- **int fprintf (FILE* stream, char* format, ...):** write the C string pointed at by format to the **stream**, using any additional arguments to fill in **format** specifiers.

Note about Library Functions

- These functions can return error codes
 - `malloc` could fail
 - a file couldn't be opened
 - a string may be incorrectly parsed
- Remember to check for the error cases and handle the errors accordingly
 - may have to terminate the program (eg. `malloc` fails)
 - may be able to recover (user entered bad input)



NYU

Backup



Byte Ordering

So, how are the bytes within a multi-byte word ordered in memory?

Conventions

- Big Endian: Sun, PPC Mac, Internet
 - Least significant byte has highest address
- Little Endian: x86, ARM processors running Android, iOS, and Windows
 - Least significant byte has lowest address

Byte Ordering

Example

- Variable x has 4-byte value of 0x01234567
- Address given by &x is 0x100

Big Endian

		0x100	0x101	0x102	0x103		
		01	23	45	67		

Little Endian

		0x100	0x101	0x102	0x103		
		67	45	23	01		

Representing Strings

Strings in C

- Represented by array of characters
- Each character encoded in ASCII format
 - Standard 7-bit encoding of character set
 - Character "0" has code 0x30
 - Digit i has code 0x30+i
- String should be null-terminated
 - Final character = 0
- Compatibility
 - **Byte ordering not an issue**

